

Instructions

Language: Python

- The totality of your source files, except all useless files (binary, temp files, cache files, `__pycache__`, `.pyc` files, etc.), must be included in your delivery.
- Don't push your main function into your delivery directory, we will be adding our own. Your files will be compiled/run with our main.py file.
- You are only allowed to use the **print()** function to complete the following tasks.
- If one of your files prevents the program from running, the evaluation will not be possible and you will receive a 0.

Runtime errors, exceptions, and crashes are all grounds for disqualification!

Delivery Instructions

Create a public repository on GitHub or GitLab containing your completed work. When you have finished all assignments, please send us the repository link for evaluation. Make sure your repository includes all source files (.py files) and follows the delivery paths specified in each assignment (e.g., `rush-1-1/*`, `rush-1-2/*`, etc.).

Project Description

The goal of this project is to display a square on the screen. In each assignment, the squares will look differently (see below).

Important: This exercise is entirely terminal-based. All output should be displayed in the command line/terminal using character-based graphics.

You have to write the **rush** function, which will be called by our main function, which will look like this:

```
def rush(x, y):
    # Your implementation here
    pass

if __name__ == "__main__":
    rush(5, 5)
```

In case of error, the function should print **Invalid size** on the standard error output and return.

Example:

```
import sys

def rush(x, y):
    if x <= 0 or y <= 0:
        print("Invalid size", file=sys.stderr)
        return
    # Your implementation here
```

All assignments must be done.

ASSIGNMENT 1

Delivery: rush-1-1/*

When calling **rush(5, 3)**:

```
o---o
|   |
|   |
o---o
```

When calling **rush(5, 1)**:

```
o---o
```

When calling **rush(1, 1)**:

```
o
```

When calling **rush(1, 5)**:

```
o
|
|
|
o
```

When calling **rush(4, 4)**:

```
o--o
|  |
|  |
o--o
```

A test script is available. Usage: **python rush1-1.py x y**

ASSIGNMENT 2

Delivery: rush-1-2/*

When calling **rush(5, 3)**:

```
 / *** \  
 *      *  
 \ *** /
```

When calling **rush(5, 1)**:

```
*****
```

When calling **rush(1, 1)**:

```
*
```

When calling **rush(1, 5)**:

```
*  
*  
*  
*  
*
```

When calling **rush(4, 4)**:

```
 / ** \  
 *   *  
 *   *  
 \ ** /
```

A test script is available. Usage: **python rush1-2.py x y**

ASSIGNMENT 3

Delivery: rush-1-3/*

When calling **rush(5, 3)**:

```
ABBBB
B   B
CBBBC
```

When calling **rush(5, 1)**:

```
BBBBB
```

When calling **rush(1, 1)**:

```
B
```

When calling **rush(1, 5)**:

```
B
B
B
B
B
```

When calling **rush(4, 4)**:

```
ABBA
B   B
B   B
CBBB
```

A test script is available. Usage: **python rush1-3.py x y**

ASSIGNMENT 4

Delivery: rush-1-4/*

When calling **rush(5, 3)**:

```
ABBBC
B  B
ABBBC
```

When calling **rush(5, 1)**:

```
BBBBB
```

When calling **rush(1, 1)**:

```
B
```

When calling **rush(1, 5)**:

```
B
B
B
B
B
```

When calling **rush(4, 4)**:

```
ABBC
B  B
B  B
ABBC
```

A test script is available. Usage: **python rush1-4.py x y**

ASSIGNMENT 5

Delivery: rush-1-5/*

When calling **rush(5, 3)**:

```
ABBBC
B   B
CBBBA
```

When calling **rush(5, 1)**:

```
BBBBB
```

When calling **rush(1, 1)**:

```
B
```

When calling **rush(1, 5)**:

```
B
B
B
B
B
```

When calling **rush(4, 4)**:

```
ABBC
B   B
B   B
CBBA
```

A test script is available. Usage: **python rush1-5.py x y**

Example Implementation Structure

Each assignment should be in its own directory with the following structure:

```
rush-1-1/
■■■ rush.py          # Contains your rush() function
■■■ main.py          # Test file (optional for your testing)

rush-1-2/
■■■ rush.py
■■■ main.py

...
```

Your **rush.py** file should contain:

```
import sys

def rush(x, y):
    """
    Display a square pattern based on x (width) and y (height)

    Args:
        x (int): Width of the square
        y (int): Height of the square
    """
    if x <= 0 or y <= 0:
        print("Invalid size", file=sys.stderr)
        return

    # Your implementation here
    pass
```

Good luck!